

Capstone: a Grounded Assistant, Built, Evaluated and Deployed
Unit 11 of the Generative AI and LLM crash course

GenAI crash course

2026-10-10

Contents

Session map	2
1. Terms: the term tree	2
2. Remember	3
3. Understand: the derivation	4
3.1 Tier 1 (easy): what a capstone is, and the project used here	4
3.2 Tier 2 (medium): the evaluation set comes first	5
3.3 Tier 3 (medium-hard): measure each stage, then decide when to refuse	6
3.4 Tier 4 (hard): how much can 26 questions tell you? The Wilson interval	6
3.5 Tier 5 (hardest): what changes with a real language model	8
4. Apply	8
4.1 Worked problems	8
4.2 Code and figures: build, evaluate, serve	8
5. Analyze	10
6. Evaluate	11
7. Create	12
8. Visualization lesson	13
9. Problem set	13
10. Retrieval practice and recap	14

Session map

Outcome of this unit

After this unit you can:

1. turn a project idea into a **design brief** with a user, a data source, a minimum viable version and **one number** that says whether it works;
2. build an end-to-end **document assistant** (chunk, retrieve, select, check, answer with a source or **refuse**) and say which stage is responsible for each wrong answer;
3. design an **evaluation set** with answerable and unanswerable questions, split it into development and test halves, and report results with **confidence intervals**;
4. choose a **refusal threshold** from a stated cost of each kind of mistake, and say why the choice made on a development set must be reported on a separate test set;
5. wrap the assistant in a **validated service**, measure its latency, and write the limits of the system in a short report.

- **Prerequisites:** Unit 4 (context windows, chat history, prompts), Unit 5 (chunking, BM25, fusion, the RAG pipeline), Unit 8 (a loop with tools, for the stretch goals), Unit 9 (groundedness, abstention, confidence intervals), Unit 10 (a validated web service, latency percentiles).
- **Difficulty ladder used:** easy → medium → medium-hard → hard (Sections 3.1 to 3.4); the hardest tier is posed as open questions in Section 3.5.
- **Deliverables:** this document (.tex, .pdf, .docx), 5 figures, the module `codes/capstone/campus_assistant/` (a handbook of a **fictional** college, 26 labelled questions, the assistant, the evaluation and diagnosis functions, a request handler, an optional FastAPI service, 6 tests), the Lab 6 chatbot module `codes/chatbot/chatbot_np/` (3 tests), and the lab track page of the course site.
- **Source anchors:** Lewis et al. 2020, arXiv:2005.11401 (retrieval-augmented generation, introduced in Unit 5; the ID was checked there against the paper); Wilson 1927 for the interval of Section 3.4 (derived here from the normal approximation, and checked against the code of Unit 9). The **handbook, the questions and every number in this unit are produced by the scripts in this unit's figures/ folder**; the college does not exist.
- **What is measured here, and what is not.** The assistant has **no language model**: its generator quotes a sentence. That makes it fully reproducible and grounded by construction, and it also means the results say **nothing about how a real LLM would score** on the same questions. The test set has 26 questions, so every rate has a wide interval (Section 3.4 and Fig. 4). Latencies are those of this toy on this CPU.

1. Terms: the term tree

Term	Symbol	One-line meaning	Where it appears	New / known
capstone project		one project that combines the units into a working system	3.1	new
design brief		one page: user, data, minimum version, metric, risks	3.1	new
minimum viable version		the smallest system that answers the real question	3.1	new
answerable / unanswerable question		the handbook does / does not contain the answer	3.2	new
development set, test set		questions used to choose settings / to report the result	3.2	new
stage-wise evaluation		measure each pipeline stage, not only the final answer	3.3	new
refusal, false refusal		the system declines / declines a question it could have answered	3.3	new
coverage		share of the question's content words found in the top chunk	3.3	new
Wilson interval		a confidence interval for a proportion that works for small n (met in Unit 9)	3.4	known
chunk, BM25, RRF		text piece; keyword ranking; rank fusion	3.3	known
groundedness		every claim in the answer is supported by the retrieved text	3.3	known
p95 latency, validation		slow-tail time; reject malformed requests	4.2	known

Terms *used but not taught here*: how to build a real vector database service, user authentication, and the legal side of storing personal data.

2. Remember

Capstone and design brief

A **capstone project** is one project that must use several units together. Start it with a **design brief** of one page: who the **user** is, which **data** they have, the **minimum viable version** (the smallest system that is useful), the **one metric** that tells you it works, and the main **risk**. Write the brief *before* the code; if you cannot name the metric, you cannot tell when you are done.

Answerable, unanswerable, development, test

A question is **answerable** if the documents contain the answer and **unanswerable** if they do not. A good evaluation set has both, because a system that never says “I do not know” is not

safe. Split the questions into a **development set** (you look at it, and tune settings on it) and a **test set** (you look once, at the end, to report the result).

The four outcomes on an answerable question

Correct; **wrong** (an answer that does not contain the fact); **refused** (the system declined a question it could answer: a **false refusal**); and, on an unanswerable question, either a **correct refusal** or **answered anyway**, which is the hallucination case.

Wilson interval

For k successes in n trials, with $\hat{p} = k/n$ and $z = 1.96$ for 95%, the **Wilson interval** is

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}. \quad (1)$$

It is the set of true rates p that the data cannot rule out. It is derived in Section 3.4.

Memory hooks.

- *Brief before build*: no metric, no project.
- *Test once*: a test set you tuned on is only another development set.
- *Say “I do not know” on purpose*: refusal is a feature that needs its own measurement.
- *Small n , wide bars*.

Recall quiz (answers in Section 10).

1. What five things go into a design brief?
2. Why must an evaluation set contain unanswerable questions?
3. A system is right on 18 of 20 questions. Is “90% accurate” a safe thing to claim?
4. What is the difference between a false refusal and a correct refusal?
5. Why is the stage that gave the wrong sentence a different problem from the stage that fetched the wrong chunk?

3. Understand: the derivation

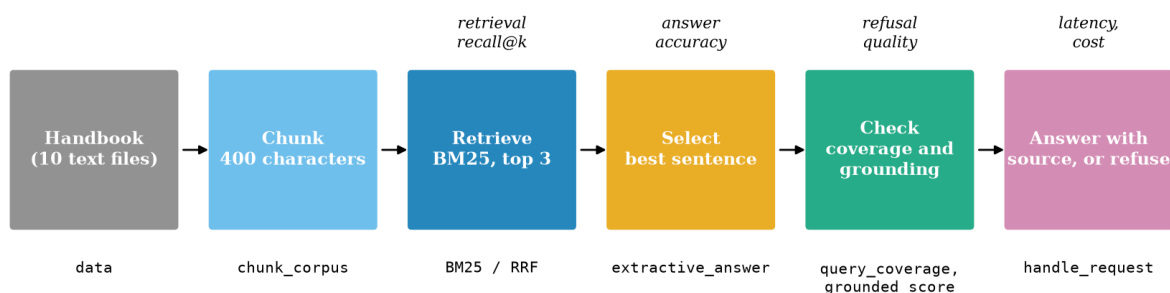
3.1 Tier 1 (easy): what a capstone is, and the project used here

Every earlier unit taught one idea. A capstone makes you combine them under realistic pressure: the data are messy, the questions are not the ones you imagined, and “it works” has to be shown with a number. Here the project is the first of the ten ideas in the syllabus: a **college information assistant**. A student asks “When does the application window close?” and the system answers from the college handbook, names the file it used, and says “I cannot answer that from the handbook” when the handbook does not say.

The college, *Riverbend Institute of Technology*, is **invented**; its handbook is ten short text files (admissions, fees, hostel, library, attendance, examinations, placements, scholarships, transport, computing),

3,074 characters in total. A handbook this small is enough to see every idea of the unit, and small enough that you can read all of it and check every answer yourself. It is *not* enough to say how a system would do on a real 300-page handbook (Section 6).

The system has six stages (Fig. 1): the handbook is **chunked** (Unit 5), the question retrieves the top three chunks with **BM25**, the **best sentence** of those chunks is selected as the answer, a **check** decides whether the evidence is good enough, and the system returns the sentence **with its source file** or **refuses**. Nothing in it is a language model: the generator only quotes. That is a deliberate teaching choice. A quoting generator cannot invent a number, so every failure you see belongs to retrieval, selection or the refusal rule, and you can find it. In Section 3.5 and Section 6 I say what changes when you put a real LLM in the last stage.



Every stage can fail on its own, so every stage gets its own measurement (Section 3.3).

The pipeline as six stages. The italic labels show which stage each number of the evaluation measures: the final accuracy alone would hide which stage failed.

3.2 Tier 2 (medium): the evaluation set comes first

Before any tuning I wrote **26 questions** against the handbook: 20 **answerable** and 6 **unanswerable**. For each answerable question I stored a **gold phrase** (the exact text in the handbook that contains the answer) and a **key** (the fact that a correct answer must contain, such as 30 April). The tests check that every gold phrase and key really occurs in the handbook. Two of the 20 questions are **paraphrases** with little word overlap with the handbook (“What is the cost of staying in a shared room for a year?” where the handbook says “a shared room costs 42000 rupees per year”); the 6 unanswerable ones are things a student would plausibly ask but the handbook never says (“Who is the principal?”, “What is the phone number of the admissions office?”).

Two rules protect the result from my own wishful thinking.

1. **Questions before system.** The questions were written, and the handbook frozen, before the first run. Changing the handbook to make a question pass would be fitting the data to the test.
2. **Development and test halves.** In Section 4 the refusal threshold is chosen on one half (every second question of each kind: 10 answerable and 3 unanswerable) and reported on the other half (the remaining 10 and 3). A threshold chosen on the numbers you report will always look better than it is.

Note the size of the unanswerable half: **three questions** per half. A single question changes a refusal

rate by 33 percentage points. I come back to this in Section 3.4; it is the most important caveat of the unit.

3.3 Tier 3 (medium-hard): measure each stage, then decide when to refuse

Stage-wise measurement. Let A be the event “the top-ranked chunk contains the gold phrase” (retrieval worked), and C the event “the answer contains the key”. By the chain rule of probability, over the answerable questions,

$$P(C) = P(A)P(C | A) + P(\bar{A})P(C | \bar{A}). \quad (2)$$

Here $P(C | \bar{A})$ is almost zero for a quoting generator (it cannot find a fact that is not in the chunk it was given), so $P(C) \approx P(A)P(C | A)$. In the first configuration of Table 1, $P(A) = 19/20$ and $P(C | A) = 18/19$, so $P(C) = 18/20 = 0.90$. Writing the product shows *where* the 10% is lost: one question in twenty at retrieval, one in nineteen at selection. These are exact counts, not an independence assumption.

The refusal rule. The check stage computes **coverage**, the share of the question’s content words (words other than the stop words of Unit 5) that occur in the top chunk:

$$\text{cov}(q, c) = \frac{|W(q) \cap W(c)|}{|W(q)|}, \quad (3)$$

where $W(\cdot)$ is the set of content words. The system refuses if $\text{cov} < \tau$. Why should coverage be a good signal? If the handbook answers the question, the chunk with the answer usually shares most of the question’s content words; if the handbook is silent, the best chunk shares few. On the 26 questions this holds: coverage has median 0.67 (range 0.33 to 1.00) for the answerable ones and median 0.25 (range 0.00 to 0.50) for the unanswerable ones, and the probability that a random answerable question has higher coverage than a random unanswerable one is 0.95 (the AUC of Unit 9). The two groups overlap in $[0.33, 0.50]$, so **no threshold is perfect**: that overlap is exactly where the mistakes will be.

Choosing τ from costs. Let c_H be the cost of answering an unanswerable question (a hallucination reaching a student) and c_F the cost of refusing an answerable one (a false refusal). Suppose that, for one question, the evidence says the handbook has the answer with probability π . The expected cost of answering is $(1 - \pi)c_H$ and of refusing is πc_F . Answer when answering is cheaper:

$$\begin{aligned} (1 - \pi)c_H &< \pi c_F && \text{(compare the two expected costs)} \\ c_H - \pi c_H &< \pi c_F && \text{(expand the left side)} \\ c_H &< \pi(c_F + c_H) && \text{(collect the terms in } \pi) \\ \pi &> \frac{c_H}{c_H + c_F}. && \text{(divide by the positive number } c_F + c_H) \end{aligned} \quad (4)$$

If a wrong answer costs five times as much as a refusal ($c_H = 5c_F$), answer only when $\pi > 5/6 \approx 0.83$. If both mistakes cost the same, answer when $\pi > 1/2$. The probability π has to be estimated from data; in this unit I do not model it and instead sweep τ and count both kinds of mistakes, which is what Fig. 3 shows. The costs are *your* decision, not a property of the data: a medical or legal assistant has a much larger c_H than a campus helper.

3.4 Tier 4 (hard): how much can 26 questions tell you? The Wilson interval

A measured accuracy $\hat{p} = k/n$ is an estimate of an unknown true rate p . If the questions are independent and each is right with probability p , then $\hat{p}$ has mean p and variance $p(1 - p)/n$, and for moderate n it

is approximately normal. With $z = 1.96$,

$$|\hat{p} - p| \leq z\sqrt{p(1-p)/n}$$

holds for 95% of samples. The textbook “Wald” interval replaces p on the right by $\hat{p}$; for $n = 20$ and $\hat{p} = 0.9$ this gives 0.9 ± 0.13 , whose upper end is 1.03, above 1, and which collapses to a point at $\hat{p} = 1$. The **Wilson interval** instead keeps p and solves the inequality for p . Square it:

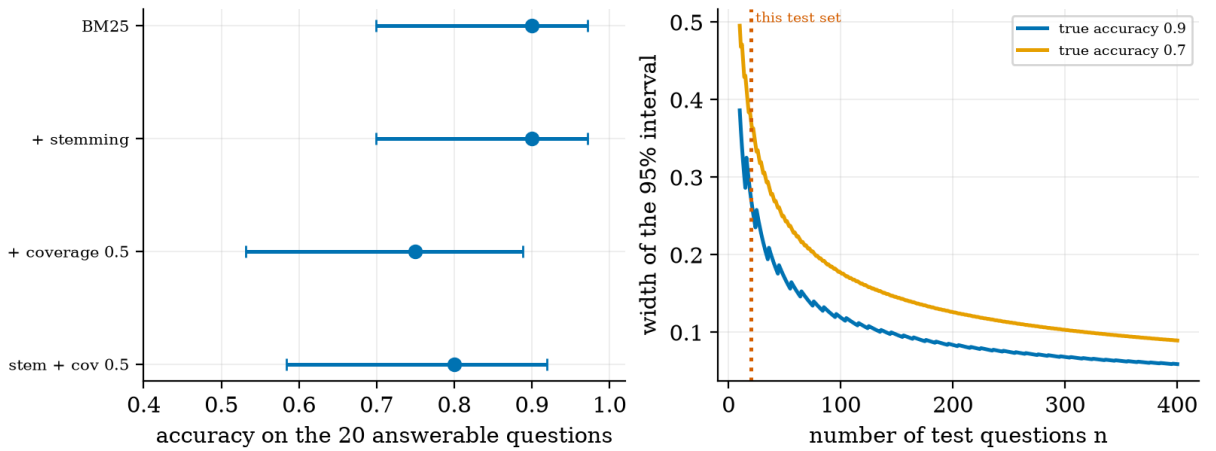
$$\begin{aligned} (\hat{p} - p)^2 &= \frac{z^2}{n} p(1-p) \quad (\text{boundary of the inequality}) \\ \hat{p}^2 - 2\hat{p}p + p^2 &= \frac{z^2}{n} p - \frac{z^2}{n} p^2 \quad (\text{expand both sides}) \\ \left(1 + \frac{z^2}{n}\right)p^2 - \left(2\hat{p} + \frac{z^2}{n}\right)p + \hat{p}^2 &= 0 \quad (\text{collect powers of } p). \end{aligned}$$

This is a quadratic in p . Its two roots are the ends of the interval:

$$p = \frac{(2\hat{p} + \frac{z^2}{n}) \pm \sqrt{(2\hat{p} + \frac{z^2}{n})^2 - 4(1 + \frac{z^2}{n})\hat{p}^2}}{2(1 + \frac{z^2}{n})}.$$

Simplify the discriminant: $(2\hat{p} + \frac{z^2}{n})^2 - 4(1 + \frac{z^2}{n})\hat{p}^2 = 4\hat{p}\frac{z^2}{n} + \frac{z^4}{n^2} - 4\hat{p}^2\frac{z^2}{n} = 4\frac{z^2}{n}\hat{p}(1-\hat{p}) + \frac{z^4}{n^2}$. Taking its square root, $2z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}$, and dividing numerator and denominator by 2 gives (1). The centre of the interval, $(\hat{p} + z^2/2n)/(1 + z^2/n)$, is pulled toward 1/2 and the interval never leaves $[0, 1]$.

For 18 correct out of 20 the interval is **[0.70, 0.97]**: with this many questions, the data cannot tell 70% from 97%. Fig. 4 shows the consequence. Moving from the plain pipeline (18/20) to one with a refusal rule (15/20, interval $[0.53, 0.89]$) *looks* like a loss of 15 points, but the intervals overlap, so **the evidence does not separate them**; for comparing two systems on the same questions use a paired comparison (Unit 4, `paired_bootstrap_ci`), which is sharper than comparing two intervals, but it cannot create information that 26 questions do not contain. For the width of the 95% interval at $\hat{p} = 0.9$ to fall to 0.10 or below you need about $n = 150$ questions (0.119 at $n = 100$, 0.1004 at $n = 140$, 0.097 at $n = 150$, 0.084 at $n = 200$).



Left: accuracy with its 95% Wilson interval for the four configurations; the intervals overlap heavily. Right: the width of the interval against the number of test questions, for true accuracy 0.9 and 0.7; the dotted line marks this unit’s 20 answerable questions (width 0.27 at 0.9).

3.5 Tier 5 (hardest): what changes with a real language model

Three open questions, which this unit cannot answer because no language model was run here.

1. **Groundedness is no longer free.** The quoting generator is grounded by construction: the Unit 9 `grounded_score` was 1.0 for all 26 answers, so the *grounded check never fired*. With an LLM that rewrites the context, the score becomes informative: for example the sentence “The annual tuition fee for B.Tech is 150000 rupees” against the handbook scores 0.0 (the number 150000 is not in the context) and a vague paraphrase scores 0.57. How good is such a lexical check against a judge model (Unit 9), measured with kappa on your own questions?
2. **Selection becomes generation.** The *wrong sentence* failures of Fig. 2 (the right chunk, the wrong sentence) are replaced by a reader that can combine two sentences, and also misread them. Does the model quote the number it should?
3. **The refusal threshold moves.** A model that is asked to answer *only from the context* will sometimes refuse on its own and sometimes not. Whether coverage still separates answerable from unanswerable questions has to be re-measured with the same development and test protocol.

4. Apply

4.1 Worked problems

Problem 1: what each stage costs. In the plain pipeline, 19 of the 20 answerable questions retrieve the gold chunk first and 18 are answered correctly. Give the stage losses. *Solution.* Retrieval loses $1/20 = 5\%$. Selection loses $1/20 = 5\%$ more (one question has the right chunk but a wrong sentence). With light stemming, retrieval loses nothing, but selection now loses 2 of 20: fixing one stage exposed the next (Fig. 2, second row).

Problem 2: a refusal rule from costs. A wrong answer is judged 4 times as bad as a refusal, so $c_H = 4c_F$. At what π do you answer? *Solution.* $\pi > c_H/(c_H + c_F) = 4/5 = 0.8$ by (4).

Problem 3: an interval by hand. 15 of 20 correct: $\hat{p} = 0.75$, $n = 20$, $z^2 = 3.84$. Centre: $(0.75 + 3.84/40)/(1 + 3.84/20) = 0.846/1.192 = 0.710$. Half-width: $1.96\sqrt{0.75 \cdot 0.25/20 + 3.84/1600}/1.192 = 1.96\sqrt{0.009375 + 0.0024}/1.192 = 1.96 \cdot 0.1085/1.192 = 0.178$. Interval $[0.53, 0.89]$, matching Fig. 4.

4.2 Code and figures: build, evaluate, serve

The whole assistant is one small class; the central lines are:

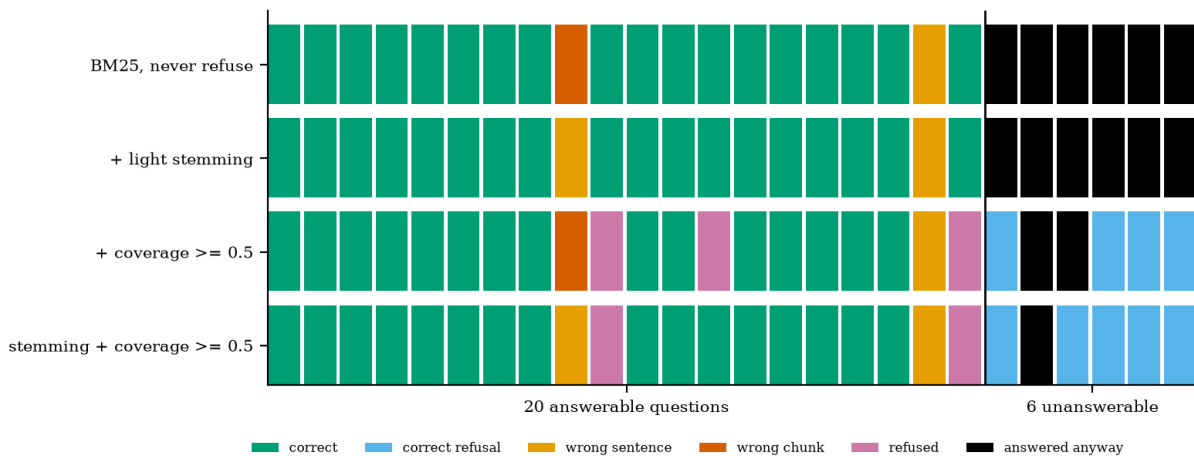
```
import campus_assistant as C
hb, qs = C.load_handbook(), C.load_queries()          # 10 files, 26 labelled questions
bot = C.Assistant(hb, chunk_size=400, k=3, cover_min=0.5, stem=True)
print(bot.ask("When do scholarship applications close?"))
# {'answer': 'Scholarship applications close on 31 July.', 'refused': None,
#  'sources': ['scholarships'], 'cover': 1.0, 'grounded': 1.0}
print(C.evaluate(bot, qs))                            # retrieval recall, accuracy, false refusal, correct
# refusal, ...
print([C.diagnose(bot, q) for q in qs])               # which stage is responsible, per question
```

Results. Table 1 is the output of `evaluate` for four configurations on all 26 questions. Retrieval is easy here because there are only ten chunks (recall@3 is 1.00 in every row, which says little); the

informative columns are recall@1, accuracy and the refusal columns.

results over the 20 answerable questions (first four columns of rates) and the 6 unanswerable ones (last column). The fraction “wrong” is the share of answerable questions answered with a sentence that lacks the fact. Same data and code as Fig. 2.

configuration	recall@1	accuracy	wrong	false refusal	correct refusal
BM25, never refuse	0.95	0.90	0.10	0.00	0.00
+ light stemming	1.00	0.90	0.10	0.00	0.00
+ coverage ≥ 0.5	0.95	0.75	0.10	0.15	0.67
stemming + coverage ≥ 0.5	1.00	0.80	0.10	0.10	0.83

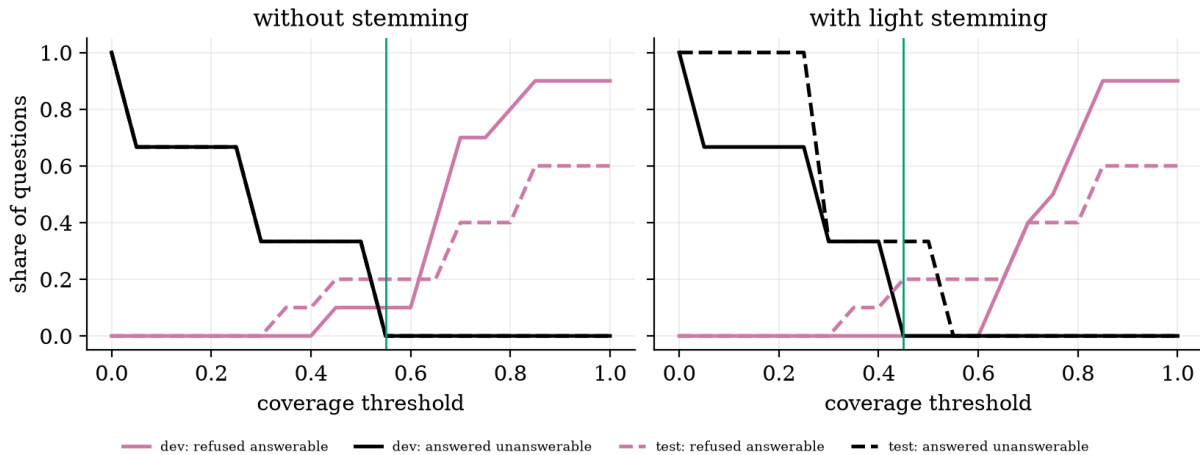


Each square is one question; each row is one configuration. Green: correct. Blue: correct refusal. Orange: right chunk, wrong sentence. Red: wrong chunk. Purple: refused although answerable. Black: answered an unanswerable question. Without a refusal rule all six unanswerable questions are answered (black); with it, five of six are refused once stemming is on.

Read Fig. 2 column by column. The red square in the first row is the question “What are the library hours on Saturday?”: the handbook says *Saturdays* and the question says *Saturday*, so plain BM25 ranks the computing-centre chunk (“Monday to Saturday”) first. Light stemming (dropping a final plural *s*) repairs the retrieval, and the square turns orange: now the library chunk is retrieved, but the sentence selected as the answer is another one from it. Fixing a stage moved the failure to the next stage, which is the reason for measuring stages separately.

Choosing the threshold honestly. Fig. 3 sweeps τ from 0 to 1 on the development half (solid) and shows the same curves on the test half (dashed). Giving a false refusal and a hallucination equal weight, the development half picks $\tau = 0.55$ without stemming and $\tau = 0.45$ with it. Without stemming, the test half then refuses 20% of its answerable questions and answers none of the unanswerable ones; **with stemming the test half still refuses 20% of answerable questions, but now answers 33% of the unanswerable ones (1 of 3)**, although the same threshold made no mistake on the development half. That gap between development and test is what a single flat number on 13 questions hides.

Serving it. `handle_request` validates the payload (a non-empty string of at most 500 characters, otherwise status 422) and returns the answer, the sources and the scores; `app_fastapi.py` exposes it as POST `/ask` and GET `/health` exactly as in Unit 10 (the test used FastAPI’s test client in this



Share of questions in each failure mode against the coverage threshold. Solid: development half (10 + 3 questions); dashed: test half (10 + 3). The green line is the threshold chosen on the development half. Each step of a curve is one question.

environment and returned 200 for a valid question and 422 for an empty one). Fig. 5 shows the time per question measured **in this process on this CPU**: BM25 alone had a median of 0.29 ms and a p95 of 0.35 ms over 780 calls; adding the LSA ranker fused by RRF raised these to 0.34 ms and 0.39 ms. It is a measurement of a toy; a real service would be dominated by the language model, not by retrieval (Unit 10).

5. Analyze

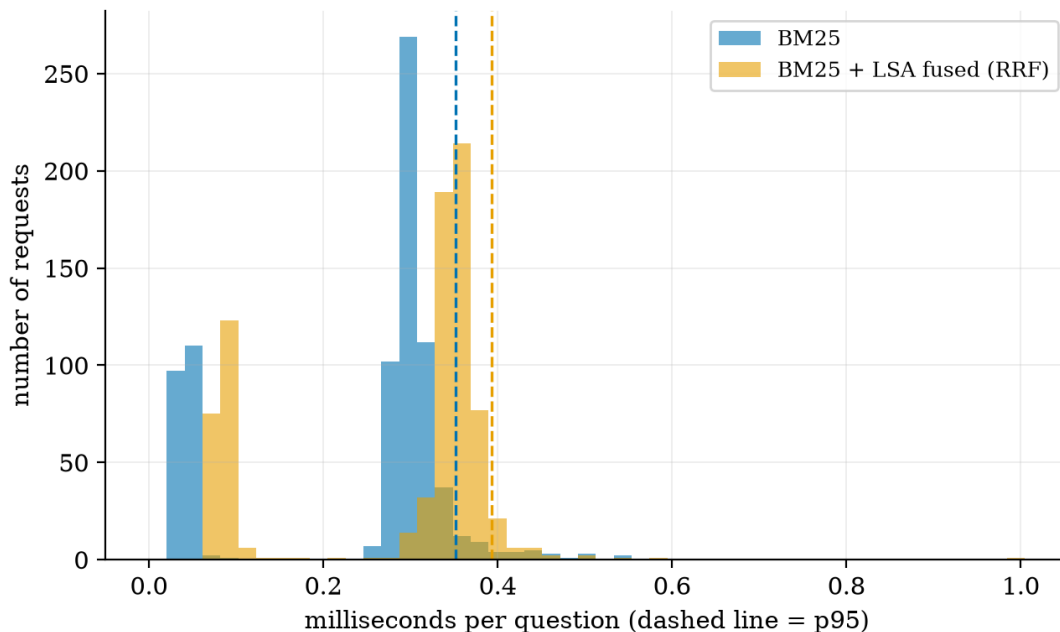
Limits and dominance. At $\tau = 0$ the system never refuses: every unanswerable question is answered (black row, 6 of 6). As τ approaches 1, only questions whose best chunk contains *every* question word survive and most answerable questions are refused (Fig. 3, right edge: 90% of the development answerable questions). The useful range is the middle, where the two failure curves cross; because coverage separates the groups only with AUC 0.95, no threshold removes both.

Which stage dominates? With stemming, retrieval at rank 1 is perfect (20 of 20) and the entire remaining error before refusal is **sentence selection** (2 of 20). So the next engineering step is not a better retriever; it is a better selector or a real reader. Without the stage-wise view you might have spent the week on retrieval.

Sensitivity to the data. I used one chunk size (400 characters) and, because every file is shorter than that, each file is exactly one chunk (the ten chunk lengths are 234 to 397 characters). The chunking decision of Unit 5 is therefore *untested* here. A real handbook with long documents would make chunk size and overlap the first thing to tune.

Failure modes seen: vocabulary mismatch (*Saturday* and *Saturdays*; the cost-of-a-shared-room question matching the guarantee sentence instead of the price sentence); refusals on questions whose wording differs from the handbook (“How many books can I borrow and for how long?” has coverage 0.40); and unanswerable questions that share generic words with a chunk (“Who is the principal of the college?” has coverage 0.50 with the transport chunk and is answered with the bus routes).

Uncertainty. Every rate in Table 1 rests on 20 or 6 questions. Fig. 4 shows 95% intervals of width 0.27 at 20 questions; the refusal rates on 6 (or 3) questions are even wider. Statements such as “stemming



Histogram of the time per question (780 calls each); dashed lines are the 95th percentiles. Retrieval on ten chunks costs a fraction of a millisecond; measure again with your own data and with the real generator.

fixes refusal” are *observations on this set*, not findings.

6. Evaluate

Alternatives. Replace BM25 by dense embeddings (Unit 5): this helps paraphrases such as “cost of staying in a shared room”, but fusing the LSA ranker with BM25 changed neither recall@1 nor accuracy here (hybrid and BM25 rows were identical), because ten chunks leave no room to improve. Replace the quoting generator by an LLM: it can combine sentences and answer in the student’s words, but it can also add what the handbook does not say, so the groundedness check of Unit 9 becomes necessary rather than decorative. Replace the coverage rule by a trained classifier on question and chunk: it can use more signals, but needs labelled data you do not have at the start.

Context. A college helper is a *low-stakes* use; a medical-literature assistant or a legal-document system (two of the other ideas) have far larger c_H in (4) and must refuse more, cite every claim, and carry a visible statement that they are not advice. Systems that touch personal data (a resume matcher) add the privacy and fairness issues of Unit 9: evaluate error rates by group and do not log what you do not need.

When not to use this design. Not for questions that need arithmetic across documents, for handbooks that change daily without re-indexing, or when the answer must be exactly right: a retrieval system with a refusal rule reduces wrong answers, it does not remove them.

Reproducibility. Everything here runs offline in seconds with NumPy; the figure scripts print every number quoted. Re-running `fig05_latency.py` gives slightly different milliseconds (machine and load); all other numbers are deterministic.

7. Create

Your capstone: pick one brief, build the smallest version first

Choose a brief from Table 2, write the one-page design brief of Section 2, then follow this order: (1) collect the documents and write 30 to 50 questions, a quarter of them unanswerable, **before** coding; (2) build the minimum version with the code of this unit; (3) split the questions, tune on one half, report on the other with intervals; (4) diagnose failures by stage and fix the dominant stage only; (5) wrap it in a validated service and measure p50 and p95; (6) write the report (below).

the ten capstone ideas from the syllabus, each reduced to a minimum version and a metric. These are design suggestions, not results.

brief (syllabus idea)	data you need	minimum viable version	main metric	main risk
College information RAG chatbot	handbook, circulars	this unit's assistant	accuracy and refusal on 40 questions	outdated documents
Research-paper assistant	20 open-access papers	answer with page and quote	citation correctness	invented references
AI coding assistant	one repository	answer from code and docs with file path	answers that run	hallucinated functions
Resume analysis and job matching	synthetic resumes only	rank resumes for a job text	agreement with human ranks	bias, personal data
Medical-literature assistant	open abstracts	quote the abstract, never advise	groundedness	being read as advice
Legal-document Q&A	public statutes	quote clause with number	exact-clause hit rate	confident wrong reading
Personalised learning assistant	course notes	explain from notes, quiz	quiz accuracy gain	stale or wrong notes
Multimodal educational tutor	figures plus captions	answer about a figure from its caption	answer match	wrong figure retrieved
Financial-document analysis	public reports	quote a figure with its page	numeric exactness	wrong period or units
Examination question generator	syllabus text	questions with answer key	expert accept rate	duplicates, wrong keys

Report and grading. The syllabus gives the capstone 10% of the course. A report of 4 to 6 pages with these parts, and this suggested weighting (my suggestion, not part of the syllabus): problem, users and data (10); system design and what was reused (15); **evaluation set, split, intervals** (30); **failure analysis by stage** and fixes (20); safety, privacy and limits, including what the system refuses (10); deployment and latency (5); reproducibility, a README that lets a stranger rerun it (10).

Combine with earlier units. Add the tool loop of Unit 8 so that the assistant can call a fee

calculator; add the judge of Unit 9 to score answers a real LLM writes; add the monitoring of Unit 10 (log the question length and the share of refusals each day and watch for drift).

8. Visualization lesson

charts taught here. The new chart is the **outcome grid**: it keeps every item visible (26 squares) instead of averaging them into one bar, so you can see that fixing retrieval moved a failure to the selection stage.

Chart type	Best for	How it misleads	matplotlib call
pipeline diagram (boxes and arrows)	which stage does what	implies every stage is equally important	<code>FancyBboxPatch,</code> <code>FancyArrowPatch</code>
outcome grid (one coloured square per item)	seeing which items fail, and how failures move between configurations (Fig. 2)	colours need a legend; order of items can create fake patterns	<code>ax.add_patch(plt.</code> <code>Rectangle(...))</code>
paired curves, development against test	whether a chosen setting generalises (Fig. 3)	steps are single questions; do not read smooth trends	<code>ax.plot(x, y);</code> <code>ax.plot(x, y, "--")</code>
point with interval bars (dot plot)	comparing estimates with their uncertainty (Fig. 4 left)	overlapping bars do not prove “no difference”	<code>ax.errorbar(x, y,</code> <code>xerr=...)</code>
interval width against n	how much data you need (Fig. 4 right)	assumes independent questions	<code>ax.plot(n, width)</code>
histogram with percentile lines	the latency tail (Fig. 5)	an average line hides it	<code>ax.hist(...);</code> <code>ax.axvline(np.</code> <code>percentile(a, 95))</code>

9. Problem set

Graded exercises

1. (easy) A test set has 40 questions and the system answers 34 correctly. Give the accuracy and say why the number alone is not enough to claim “85%”.
2. (medium) Of 12 answerable questions, a system answers 9 correctly, refuses 2 and answers 1 wrongly. Give the three rates and check that they add to 1.
3. (medium-hard) Using (4), at what probability π should a legal assistant answer if a wrong answer is judged 9 times as costly as a refusal? Compare with the campus helper of Problem 2.
4. (hard) Compute the Wilson interval for 34 of 40 with (1) (or `evaluation_np.wilson`), and use the code to find the smallest n (in steps of 10) for which a measured 90% has a 95% interval no wider than 0.10.
5. (hardest) Write the evaluation design for the *resume analysis* brief: what is an “unanswerable” item there, which groups need separate error rates, what is the unit of the test split (resume or person), and what must never appear in the logs?

10. Retrieval practice and recap

Answers to the Section 2 quiz.

1. The user, the data, the minimum viable version, the one metric, and the main risk.
2. Because a system that always answers cannot be told apart from one that knows when to stop; only unanswerable questions measure refusal and hallucination.
3. No: with 20 questions the 95% Wilson interval is $[0.70, 0.97]$; state the interval and the number of questions.
4. A false refusal declines a question the documents could answer (a lost answer); a correct refusal declines one they cannot (a prevented hallucination).
5. A wrong chunk is a retrieval problem (ranking, vocabulary, chunking); a wrong sentence with the right chunk is a selection or reading problem. They have different fixes.

Answers to the Section 9 exercises.

1. $34/40 = 0.85$. The interval is wide (see 4), and a point estimate on 40 questions does not support “85%”.
2. $9/12 = 0.75$ correct, $2/12 \approx 0.167$ refused, $1/12 \approx 0.083$ wrong; the sum is 1.
3. $\pi > 9/10 = 0.90$ (campus helper in Problem 2: 0.80). A costlier mistake demands more confidence before answering.
4. $[0.71, 0.93]$, width 0.22 (computed with the code). Width at 90%: 0.119 at $n = 100$, 0.1004 at $n = 140$ (just above 0.10), 0.097 at $n = 150$; the smallest n on the grid is **150**.
5. A sketch: unanswerable = a resume with no evidence for the required skill (the system should say “not shown”); report error rates by gender, college and language of the resume; split by *person*, never by resume (the same person in both halves leaks); never log names, contact details or full resume text.

Explain it to a friend.

- Why can a system with 90% accuracy still be unsafe?
- Why do you split the questions into two halves before choosing a threshold?
- Why does fixing the retriever sometimes make a different mistake appear?

Re-derive tomorrow without looking: (a) the answer rule (4); (b) the Wilson interval (1) from the quadratic in p ; (c) the chain (2) for the stage losses.

11. Where next

This is the capstone of the course; it ties **using models** (Units 4 to 6), **extending them** (Units 7 and 8), **trusting them** (Unit 9) and **running them** (Unit 10) into one deliverable. The lab track on the course site maps all fourteen laboratory exercises of the syllabus to the code you already have, and the capstone is Lab 14.

- **Terms met but not taught:** question-answering benchmarks beyond a hand-written set, active learning for labelling, reranking models.

- **Open threads:** how the same 26 questions score with a real language model as the reader (a notebook was not written for this unit); whether coverage still separates answerable from unanswerable questions when the generator is an LLM; the chunk-size question on a handbook with long documents.